

CHAPTER 5

TRANSACTIONS AND CONCURRENCY CONTROL

5.1 Introduction to Transactions

A transaction in the Streametrics DBMS is a logical, atomic unit of database processing that includes one or more database access operations. Due to the complex relationships in an OTT platform (e.g., adding a movie and mapping genres simultaneously), ensuring data integrity requires robust transaction handling.

5.1.1 Properties (ACID)

- **Atomicity:** Ensures that all operations in the transaction (like inserting a user and setting up their subscription) execute completely, or none at all.
- **Consistency:** The database must transition from one valid state to another, preserving constraints (e.g., ratings between 0 and 10).
- **Isolation:** Concurrent viewers modifying watch histories do not interfere with each other.
- **Durability:** Once a subscription payment is committed, it remains permanently stored even through system failures.

5.1.2 States

A transaction passes through various states: **Active** (while executing), **Partially Committed** (after the final statement), **Failed** (if an error or constraint violation occurs), **Aborted** (after a rollback finishes), and **Committed** (successful completion).

5.2 Transaction Control Language (TCL)

TCL statements manage the changes made by DML statements explicitly.

- **Savepoint:** Sets a marker within a transaction, allowing temporary partial rollbacks without abandoning entire transactions.
- **Commit:** Permanently saves all changes made during the current transaction.
- **Rollback:** Undoes all uncommitted changes, reverting the database to the state it was in before the transaction began.

5.3 Streametrics Transactions Execution

Below are 5 fully executed transaction blocks targeting the OTT architecture, utilizing Commits, Rollbacks, and Savepoints.

5.3.1 Transaction 1: New Movie Setup with Rollback Recovery

Inserting a highly anticipated movie and connecting it to the genre tables atomically.

```
MariaDB [ott_platform_db]> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

-- Step 1: Insert the movie entry
MariaDB [ott_platform_db]> INSERT INTO movies (title, release_year, duration, rating)
VALUES ('Cosmic Horizon', 2026, 140, 8.5);
Query OK, 1 row affected (0.01 sec)

-- Step 2: Establish a safe checkpoint
MariaDB [ott_platform_db]> SAVEPOINT after_movie_insert;
Query OK, 0 rows affected (0.00 sec)

-- Step 3: Accidental bad mapping (Genre 99 does not exist)
MariaDB [ott_platform_db]> INSERT INTO movie_genres (movie_id, genre_id) VALUES
(LAST_INSERT_ID(), 99);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails

-- Step 4: Oops! Error occurred, roll back to safe point
MariaDB [ott_platform_db]> ROLLBACK TO after_movie_insert;
Query OK, 0 rows affected (0.00 sec)

-- Step 5: Insert correct genres safely (1 = Action, 4 = Sci-Fi)
MariaDB [ott_platform_db]> INSERT INTO movie_genres (movie_id, genre_id) VALUES
(LAST_INSERT_ID(), 1), (LAST_INSERT_ID(), 4);
Query OK, 2 rows affected (0.01 sec)

MariaDB [ott_platform_db]> COMMIT;
Query OK, 0 rows affected (0.02 sec)
```

5.3.2 Transaction 2: User Upgrading Subscription

Safely updating a user's subscription tier from Basic to Premium.

```
MariaDB [ott_platform_db]> START TRANSACTION;

-- Deactivate old subscription
MariaDB [ott_platform_db]> UPDATE subscriptions SET status = 'inactive' WHERE user_id = 1
AND status = 'active';
Query OK, 1 row affected (0.01 sec)

-- Insert new Premium subscription
MariaDB [ott_platform_db]> INSERT INTO subscriptions (user_id, plan_name, start_date,
end_date, status) VALUES (1, 'Premium', CURDATE(), DATE_ADD(CURDATE(), INTERVAL 1 YEAR),
'active');
Query OK, 1 row affected (0.01 sec)

MariaDB [ott_platform_db]> COMMIT;
Query OK, 0 rows affected (0.02 sec)
```

5.3.3 Transaction 3: Aborting an Invalid Movie Review

A transaction where a user attempts to rate a movie negatively beyond constraints.

```
MariaDB [ott_platform_db]> START TRANSACTION;

MariaDB [ott_platform_db]> UPDATE movies SET rating = -2.5 WHERE movie_id = 5;
ERROR 4025 (23000): CONSTRAINT `CONSTRAINT_1` failed for `ott_platform_db`.`movies`
-- The DBMS blocked updating due to rating CHECK (rating >= 0 AND rating <= 10)

-- Action rejected, rolling back the entire transaction
MariaDB [ott_platform_db]> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

5.3.4 Transaction 4: Batch Logging Watch History & Sync

Logging a movie view and syncing the user's active login state simultaneously.

```
MariaDB [ott_platform_db]> START TRANSACTION;

-- Log view history at 50%
MariaDB [ott_platform_db]> INSERT INTO watch_history (user_id, movie_id,
progress_percent) VALUES (1, 2, 50);
Query OK, 1 row affected (0.01 sec)

MariaDB [ott_platform_db]> SAVEPOINT history_logged;

-- Sync user's last activity timestamp
MariaDB [ott_platform_db]> UPDATE users SET last_login = NOW() WHERE user_id = 1;
Query OK, 1 row affected (0.01 sec)

MariaDB [ott_platform_db]> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

5.3.5 Transaction 5: Removing a Flagged User

Deleting a malicious user account with a checkpoint in case they are actually an admin.

```
MariaDB [ott_platform_db]> START TRANSACTION;

-- Log the administrative action
MariaDB [ott_platform_db]> INSERT INTO notifications (user_id, type, message) VALUES (2,
'general', 'Your account is scheduled for deletion due to TOS violation.');
```

```
MariaDB [ott_platform_db]> SAVEPOINT admin_notified;

-- Delete user (Cascade will wipe their watch history)
MariaDB [ott_platform_db]> DELETE FROM users WHERE user_id = 2;
Query OK, 1 row affected (0.02 sec)

-- Wait, user 2 was falsely flagged! Revert the deletion.
MariaDB [ott_platform_db]> ROLLBACK TO admin_notified;
Query OK, 0 rows affected (0.00 sec)

MariaDB [ott_platform_db]> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

5.4 Concurrency Control

Concurrency ensures that multiple transactions can execute concurrently without violating the consistency of the database, which is vital for an OTT platform handling millions of simultaneous viewers.

5.4.1 Concurrency Control Algorithms & Locking

a. Row-Level Locking – **SELECT ... FOR UPDATE**

Locks specific rows retrieved by a query so other transactions cannot update or delete them until the lock is released with a COMMIT. Essential for high-concurrency billing.

b. Table-Level Locking – **LOCK TABLE**

Locks entire tables for bulk operations or schema maintenance.

Lock Modes:

Lock Mode	Description
ROW SHARE	Allows concurrent access; prevents other sessions from locking the table exclusively.
ROW EXCLUSIVE	Prevents other sessions from locking in share mode. Used by default for DML (INSERT/UPDATE/DELETE).

SHARE	Allows queries but not updates or deletes.
SHARE ROW EXCLUSIVE	A stricter mix; more restrictive than SHARE.
EXCLUSIVE	Prevents all other access — full table lock.

c. **COMMIT** – Releases All Locks held by the transaction.

d. **ROLLBACK** – Undoes Changes & Releases Locks held by the transaction.

5.4.2 Project Example: Handling Billing Concurrency

In Streametrics, if an automated billing service and a user upgrading their plan try to modify the subscription simultaneously, we lock the user's active subscription row explicitly to prevent a race condition.

```
-- Session A (Billing Engine) begins transaction
MariaDB [ott_platform_db]> START TRANSACTION;

-- Places a Row-Level Exclusive Lock on User 1's active subscription
MariaDB [ott_platform_db]> SELECT * FROM subscriptions WHERE user_id = 1 AND status =
'active' FOR UPDATE;
+-----+-----+-----+-----+-----+
| subscription_id | user_id | plan_name | start_date | end_date | status |
+-----+-----+-----+-----+-----+
| 1 | 1 | Premium | 2026-04-19 | 2027-04-19 | active |
+-----+-----+-----+-----+-----+

-- While Locked, if Session B tries to UPDATE this row, it gets blocked (waiting).

-- Billing Engine updates the end date safely
MariaDB [ott_platform_db]> UPDATE subscriptions SET end_date = DATE_ADD(end_date,
INTERVAL 1 YEAR) WHERE subscription_id = 1;

MariaDB [ott_platform_db]> COMMIT;
-- The lock is released. Session B can now proceed securely.
```